©SHUTTERSTOCK/BRENT WALKER

A Semi-Supervised Active Learning Neural Network for Data Streams With Concept Drift

Botao Jiao
China University of Petroleum (East China), Qingdao, 266580, CHINA and also Victoria University of Wellington, Wellington, 6140, NEW ZEALAND

Heitor Murilo Gomes *and* ***Bing Xue***
Victoria University of Wellington, Wellington, 6140, NEW ZEALAND

Yinan Guo
China University of Mining and Technology (Beijing), Beijing, CHINA

Mengjie Zhang
Victoria University of Wellington, Wellington, 6140, NEW ZEALAND

***Abstract*—**Learning from data streams originating from non-stationary environments is vital for many real-world applications. A notable challenge in this task is concept drift. Most existing methods rely on a large number of labeled instances to detect and tackle concept drift in data streams. However, obtaining labeled data is not easy due to high costs,

Digital Object Identifier 10.1109/MCI.2024.3486680
Date of current version: 9 January 2025

This article was recommended for publication by Associate Editor Alexander Wong.

Corresponding author: Yinan Guo (e-mail: nanfly@126.com).

1556-603X © 2025 IEEE.
All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies.
Authorized licensed use limited to: GITAM University. Downloaded on September 10,2025 at 07:15:08 UTC from IEEE Xplore. Restrictions apply.

especially in potentially infinite data streams. To address this issue, a semi-supervised active learning neural network for data streams with concept drift is proposed to build an accurate classification model from incompletely labeled data. First, a semi-supervised regularization based on smoothness assumption is proposed to adjust the network weights and utilize unlabeled data. Second, a smoothness loss-based query strategy is designed to select instances that effectively improve model performance in line with the semi-supervised regularization objective. Notably, the query strategy and semi-supervised regularization form a closed learning loop that realizes the mutual enhancement of semi-supervised learning and active learning. Furthermore, an adaptive node adjustment method is proposed, which adjusts only a few neurons to adapt to local changes. Experiments on 17 synthetic and real-world datasets show that the proposed approach outperforms other state-of-the-art methods under various labeling budgets.

I. Introduction

With the advancement of both sensors [1] and wireless communication technologies [2], a large volume of data is rapidly generated in the form of streams, known as data stream or streaming data [3]. Different from batch data, changes in the environment and working conditions inevitably lead to the appearance of the concept drift problem in data streams [4]. Without loss of generality, concept drift means that the joint distribution of data changes over time [5]. Once a concept drift occurs, the classifiers created from historical data cannot accurately recognize subsequent instances, leading to a decline in model performance [6]. Although ample studies have been carried out on tackling concept drift, most of them rely on fully labeled training data [7]. However, labeling all instances by humans in a potentially infinite data stream may be costly and impractical [8]. For example, the COVID-19 pandemic has brought widespread use of AI-based drug discovery for predicting potentially effective compounds. To train a prediction model, it is extremely time-consuming and costly to test the drug-protein interaction of large numbers of compounds. In addition, as the virus mutates, the drugs screened out by the AI model might no longer be effective, presenting a concept drift problem.

The existing methods for managing concept drift in data streams can be classified into two main categories, including informed and blind approaches [9]. In informed approaches, a particular drift detection module is designed to identify a drift [10]. The module finds the most appropriate time for splitting the old and new concepts and triggers an adaptation process to learn a new concept. However, a drift may only occur in the local area of feature space [11], called a regional drift. Informed approaches usually ignore valuable information from historical classifiers in the non-drifted regions [12]. Additionally, the triggering conditions of a drift alarm are typically pre-designed by humans in advance, and the inappropriate preset threshold may lead to false alarms and missed detections. Being different from informed approaches, blind ones forget old information and constantly update classifiers by the latest data without detection module [13]. In order to fully utilize the valuable historical information from non-drifted regions, reuse mechanisms of the historical classifiers are incorporated into the canonical blind approaches [11], [14]. These historical classifiers are uniformly applied across the entire feature space, introducing outdated information from drift regions.

Jiao et al. [15] designed a dynamic ensemble selection mechanism, where for each newly arrived instance, a unique ensemble is constructed based on the local performance of base classifiers. In this way, the historical base classifiers can still provide accurate predictions for instances from non-drifted regions without affecting the learning from drifted regions. This method achieves regional drift adaptation, but it relies on the ground truth of all instances for base classifier selection, which is challenging to apply in reality [16]. Thus, it makes more sense to retain valuable historical knowledge when labels are scarce, as it is often time-consuming to re-learn the complete classification boundary with a small number of labeled data [12].

To minimize labeling costs, active learning (AL) [17] and semi-supervised learning (SSL) [18] have been introduced to effectively learn from incompletely labeled data. AL focuses on selectively labeling a limited amount of data to efficiently train a classifier rather than requiring all true labels [19]. Previous studies on AL-based data stream classifiers have proven effective in improving performance. However, the valuable information embodied in many unlabeled instances is not fully utilized [20]. Conversely, SSL fully leverages the untapped potential of unlabeled data to enhance classification accuracy. However, the labeled instances are typically obtained through random labeling, making their performance susceptible to the quality of the labeled data. Based on this, integrating the advantages of AL and SSL is a natural idea [21]. Unfortunately, Mittal et al. [22] proved that a simple combination of both cannot promote the algorithm performance. Through analysis, it was found that AL is independently designed to learn from labeled instances and ignore the knowledge implied in unlabeled instances [23].

Radial basis function neural networks (RBFNNs) [24] are a commonly used artificial neural network with radial basis functions as activation functions. Typically, RBFNNs consist of only one hidden layer, utilizing a relatively small number of neurons to achieve better approximation and faster learning compared to other types of neural networks [25]. In addition, RBFNN is a local learning modeling paradigm, where each hidden neuron is mainly responsible for modeling a part of the input space [26]. The response of a neuron in the hidden layer decreases monotonically with the distance from the center of the radial basis function, signifying that adjustments to a node do not affect

Authorized licensed use limited to: GITAM University. Downloaded on September 10,2025 at 07:15:08 UTC from IEEE Xplore. Restrictions apply.

the networks predictions in other locations [27]. Apart from local learning modeling, RBFNNs can be trained by hybrid learning algorithms [28]. The centers of radial basis functions in the hidden layer are first determined to learn compact data representations, and then the decision boundary is adjusted by updating the weights of the output layer. The training of the hidden layer can be performed with labeled and unlabeled data [29]. Therefore, RBFNNs are suitable for our problem since they can adapt to local changes by adjusting only a few neurons and efficiently learn from unlabeled data.

This study introduces a semi-supervised active learning neural network for data streams with concept drift (SSALD) based on RBFNNs. First, a sliding window gathers the recent instances from a data stream, creating a snapshot of the current data distribution. An AL strategy is developed to select informative cases within a limited labeling budget, enhancing the SSL efficiency. Next, the centers of radial basis functions are adjusted to represent the data distribution within the current window. Subsequently, a regularization mechanism based on the smoothness assumption is proposed to maximize the output smoothness on neighboring instances. Finally, the output layer weights are updated using a loss function with regularization.

The main contributions of this work are as follows.

1) A novel active learning strategy is developed to provide a good learning basis for semi-supervised learning. The proposed query strategy considers the discriminative information of labeled instances while maximizing the efficiency of SSL to better propagate the knowledge of labeled instances to unlabeled data.
2) A regularization mechanism based on a smoothness assumption is proposed to exploit unlabeled data. It forces the output of the model to be smooth on neighboring instances, thereby constructing a smooth decision boundary with a small number of labeled instances.
3) An RBFNN center adjustment method is developed to tackle concept drift. It can preserve valuable discriminative information in non-drift regions while learning the new distribution from a regional drift, improving the efficiency of drift adaptation.

The rest of this paper is organized as follows. Section II defines the concept of drift and reviews the AL and SSL methods for data streams. Section III describes the principle of the proposed semi-supervised active learning neural network. Then, an introduction to the experimental design is given in Section IV, and the experimental results are compared and analyzed in Section V. At last, the whole paper is concluded and our planned future work is given in Section VI.

II. Related Work

A. Problem Statement and Definition of Concept Drift

Data stream is a sequence of instances $S = \{d_t\}_{t=1}^{T}$ [30]. If d_t is labeled, $d_t = (x_t, y_t)$, otherwise, $d_t = x_t$, where $x_t \in \mathbb{R}^d$ is a d-dimensional feature vector, and y_t is its label. Suppose a new instance arrives at each time step, following a certain distribution $P_t(x, y)$. Concept drift is defined as $\exists t, P_t(x, y) \neq P_{t+1}(x, y)$ [31]. Data stream classification algorithms aim to learn a classifier from $\{d_1, \ldots, d_t\}$ to accurately predict the label of x_{t+1}. For supervised models, the classifier can obtain y_{t+1} immediately after prediction, whereas AL and SSL models cannot obtain y_{t+1} in most cases.

As the joint probability $P_t(x, y)$ can be decomposed into $P_t(x, y) = P_t(x)P_t(y \mid x)$, concept drift can be divided into two categories [32].

- *Virtual Drift*: $P_t(x) \neq P_{t+1}(x)$ while $P_t(y \mid x) = P_{t+1}(y \mid x)$. That is, the change in the prior probability does not affect the posterior distribution. In other words, the decision boundary has not changed.
- *Real Drift*: $P_t(y \mid x) \neq P_{t+1}(y \mid x)$. Any change in the posterior probability is called true drift because it results in a new decision boundary.

The performance of the classifiers is directly affected by changes in decision boundary, so most existing research focuses on the real drift [33]. Additionally, concept drift can be categorized into three types based on the duration of changes: sudden, gradual, and incremental drift. In a sudden drift, a new concept replaces the old one within a short time, while in a gradual or incremental drift, an old concept evolves into a new one, step by step over some time. The key difference is that the incremental drift produces multiple intermediate concepts that differ from old and new ones. In real-world scenarios, the type of drift is usually determined by the physical time of the changes and expert experience. The above two types of drifts still cannot clearly be distinguished by the definitive timestamp threshold.

In general, these drift division methods focus on the time point of the change and use the change of $P(x, y)$ to represent the drift. This expression implies that the changes are global. However, concept drifts might only occur within some specific regions $\mathbb{C}^d \subset \mathbb{R}^d$. That is, $P_t(x, y \mid x \in \mathbb{C}^d) \neq P_{t+1}(x, y \mid x \in \mathbb{C}^d)$ while $P_t(x, y \mid x \in \mathbb{R}^d, x \notin \mathbb{C}^d) = P_{t+1}(x, y \mid x \in \mathbb{R}^d, x \notin \mathbb{C}^d)$, which is called regional drift [34]. The most common strategy to adapt to concept drift is to learn a new classifier for the new concept. However, this strategy might not be the best for dealing with regional drift, as the knowledge acquired prior to the drift may remain valid after a regional drift occurs.

B. Active Learning for Data Streams

To ensure accurate classification accuracy with limited labeling cost, AL is used to choose specific instances for labeling during the training of classifiers [35].

Query strategy is the critical component of AL, which selects unlabeled instances that meet specific criteria for labeling. Zliobaite et al. [19] outlined three requirements for query strategies and developed two uncertainty strategies to label instances with high classification uncertainty. However, these

Authorized licensed use limited to: GITAM University. Downloaded on September 10,2025 at 07:15:08 UTC from IEEE Xplore. Restrictions apply.

strategies focus only on instances near the classification boundary and may overlook potential changes elsewhere. Therefore, they proposed a hybrid strategy that combines prediction uncertainty and random selection to ensure the diversity of labeled data. Ienco et al. [36] proposed a chunk-based AL framework. When a new data chunk arrives, the instances are first divided into clusters using a clustering algorithm. Then, some representative instances are selected from each cluster to cover the entire feature space.

Zgraja et al. [37] introduced a weighting mechanism into the clustering algorithm, selecting more instances from higher-weight clusters, also known as a chunk-based method. Compared with the chunk-based method, the online method considers the one-by-one learning process and fits more scenarios. Liu et al. [17] developed an AL framework based on a dual-query strategy to select representative instances based on local density and uncertainty. They also designed a forgetting mechanism based on Ebbinghaus' law to deal with drift and noise. Zhang et al. [38] proposed a hybrid labeling strategy that combines the uncertainty and imbalance strategies to handle concept drift in imbalanced data streams and label more minority instances. They also designed a data buffer to provide more minority training instances. Combining ensemble learning with AL has attracted considerable attention due to its high generalization.

Alabdulrahman et al. [39] introduced an active online ensemble algorithm that extends online bagging and boosting with an AL component, where the example with the highest predicted probability is selected for labeling. Krawczyk [40] employed a naive Bayes classifier with a forgetting mechanism as the base learner to further improve drift adaptation and efficiency, enabling it to track the latest data distribution without requiring an external concept drift detector. Additionally, a one-vs-one strategy is utilized to leverage a multi-class problem into multiple two-class tasks to build a simple yet diverse ensemble model. Guo et al. [41] introduced a robust online active learning strategy with dynamic weights for unbalanced imperfect data streams. This method effectively identifies outliers and distinguishes them from concept drift, measuring the distance of each newly arrived instance from the center of each cluster, effectively mitigating the negative impact of outliers on classification performance.

These methods learn the underlying distribution through representative instances labeled by a specially designed query strategy [42]. However, in a data stream, it can be challenging for the classifier to learn new decision boundaries with only a small number of labeled instances at the beginning of a new concept. AL requires more time to collect data from new ideas and select enough representative instances for the classifier to learn from, reducing the efficiency of adapting to concept drift.

C. Semi-Supervised Learning for Data Streams

SSL methods do not actively request instances to be labeled. Instead, the assumption is that the stream (or data window) contains a small portion of labeled data. These methods use both labeled and unlabeled instances to train a predictive model. Using unlabeled data for modeling is the critical point in designing semi-supervised methods [43], [44].

The smoothness assumption is a powerful concept that suggests the instances close to each other, which tend to share a similar label, determining decision boundaries that favor low-density regions [45]. Based on this assumption, Shen et al. [46] proposed a two-layer self-organizing incremental neural network, with the first layer learning the probability density distribution of the input data and the second layer identifying different class instances by detecting the low-density area to separate different class instances. However, its ability to handle concept drift is limited. Zhang et al. [47] developed a clustering algorithm to cluster from data streams and introduced a label propagation method to infer class labels for each cluster. To address the concept drift, they designed a weighting schema to use all base models for the ensemble classifier. Soares [48] proposed RBF networks using manifold regularization to exploit unlabeled data while addressing concept drift in classification problems. Manifold regularization maximizes the smoothness of a classifier, where the smoothness is defined by summing the classifier's local variations among nearby instances. Gu [49] recently proposed a dual-model self-organizing fuzzy inference system. By integrating two cutting-edge zero-order evolving fuzzy systems, the model can continuously identify representative instances from new pseudo-labeled data and self-improve to minimize the impact of erroneous pseudo-labels.

In autonomous driving scenarios, Zhang et al. [50] utilized K-nearest neighbor-based ensemble learning to classify the maneuvering behaviors of surrounding vehicles. Their algorithm involves initializing several micro-clusters with a small amount of labeled data and employing KNN classifiers to predict the class label for each incoming instance. Notably, labeled and unlabeled data are used to update or create micro-clusters. For a more systematic and comprehensive review of AL and SSL in data streams, please refer to [51].

The SSL performance depends on the quality of labeled instances and the underlying marginal data distribution. When the labeled data cannot represent the entire distribution, the SSL help in performance improvement will be limited [52]. However, data labeling is usually independent of the model design used in the above methods. Therefore, introducing query strategies into SSL is a feasible way to improve its performance [53]. SSL embodies knowledge from unlabeled data, and AL helps the semi-supervised model better propagate the understanding of labeled instances to unlabeled data. The critical point is that the query strategy should be consistent with the goal of SSL to improve unlabeled data utilization. Contrarily, directly combining existing AL methods with SSL may be counterproductive [22].

Authorized licensed use limited to: GITAM University. Downloaded on September 10,2025 at 07:15:08 UTC from IEEE Xplore. Restrictions apply.

III. Proposed Method

The proposed semi-supervised active learning neural network for data streams with concept drift is essentially a RBFNN comprising of an input layer, a hidden layer, and an output layer. The input layer corresponds to the feature vector of the data x, while the hidden layer is composed of H nodes with Gaussian radial basis functions as activation functions, which can be formalized as follows:

$$\varphi_i = \varphi_i(x, c_i, \sigma) = \exp\left(-\|x - c_i\|^2 / 2\sigma^2\right) \tag{1}$$

where c_i and σ are the center and width of the radial basis function of the ith node, respectively. This paper focuses on binary classification, making the output layer have only one node, and the sigmoid function is used as the activation function. The network's output of the network is expressed as follows:

$$f(x) = \frac{1}{1 + \exp\left(-\sum_{i=1}^{H} \omega_i \varphi_i\right)} \tag{2}$$

where ω_i is the weight of ith node in the output layer. SSALD mainly consists of three parts: a smoothness loss-based query strategy, adaptive node adjustment and semi-supervised regularization. First, the smoothness loss-based query strategy selects informative instances from a sliding window based on feature space coverage and predictive smoothness to help spread label information. Then, the hidden nodes are adaptively adjusted to accommodate changes in data distribution caused by drift within the sliding window. Finally, the network weights are updated using a loss function with a semi-supervised regularization term, and a smooth decision boundary is learned from both labeled and unlabeled instances. Figure 1 shows the overall framework of SSALD.

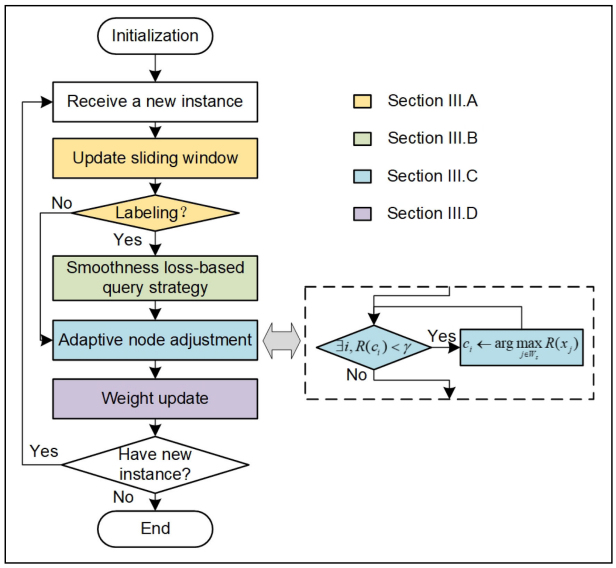


FIGURE 1 The flowchart of the proposed SSALD algorithm.

A. Sliding Windows and Labeling Budget

The sliding window is a commonly used module in data stream learning algorithms. It stores the latest instances from a data stream and provides training instances for model updates. These instances can also be seen as a snapshot of the current data distribution. Additionally, sliding windows can offer more selectable instances and information for query strategy to select more discriminative instances under a limited budget B. The budget $B \in [0, 1]$ represents the proportion of instances human experts can label, predefined based on the labeling capacity. Precisely, $B = 0$ and $B = 1$ correspond to unsupervised learning and supervised learning, respectively. The other cases follow SSL. We ensure that label cost does not exceed the preset budget B by controlling the number of labeled instances within the sliding window.

Let W_s be the sliding window containing the latest N instances $\{d_{t-N+1}, \ldots, d_t\}$. Once a new unlabeled instance $d_{t+1} = \{x_{t+1}\}$ arrives, d_{t-N+1} is removed from W_s. At each time step, SSALD triggers the query strategy to label an instance in W_s with probability

$$P = 1 - \frac{n_l}{BN} \tag{3}$$

where n_l denotes the number of labeled instances in sliding window W_s. As n_l gets closer to BN, the probability P continuously decreases until $n_l = BN$. Moreover, $P = 0$ signifies that the model will not assign a label to any instances until new data arrives to guarantee $n_l \leq BN$ at all time steps. In other words, this approach guarantees that the cost of labeling data adheres to the budget at any stage of the data stream. It is important to emphasize that while we use a sliding window to control labeling costs and provide alternative examples for query strategy, our model still learns online, processing data one by one as it arrives. Additionally, the sliding window can preserve valuable instances when the labeling budget is limited, allowing relabeling when the budget is available.

B. Smoothness Loss-Based Query Strategy

The smoothness assumption is a common concept in SSL. It suggests that the output of a classifier on adjacent instances is the same or similar [52]. By smoothing the output between nearby instances, the classifier will generate a preference for geometrically simple decision boundaries, which improves generalization. The classifier smoothness output on an unlabeled instance that can be described as the sum of the labels of nearby instances. Therefore, we can estimate the output of the classifier on unlabeled instances based on the ground truth of the labeled instances. Labeling instances from regions where the classifier output is not smooth can help spread label information propagation.

However, labeled instances within a sliding window are often insufficient when the labeling budget is limited. To provide more label information, a label window W_l is used to store the last L labeled instances. For each unlabeled instance

Authorized licensed use limited to: GITAM University. Downloaded on September 10,2025 at 07:15:08 UTC from IEEE Xplore. Restrictions apply.

x_i in the sliding window, the smoothness label can be expressed as follows:

$$s_i = \frac{\sum_{j \in W_l} w_j S_{ij} y_j}{\sum_{j \in W_l} w_j S_{ij}} \tag{4}$$

where w_j is weight of jth instance x_j in W_l and S_{ij} is the similarity between x_i and x_j. The weight of the most recent labeled instance is initialized to 1 and then decremented with the arrival of new labeled instances as $w_j \leftarrow w_j(1 - 1/L)$. The similarity is obtained based on the RBF kernel as follows:

$$S_{ij} = \exp\left(-\left\|x_i - x_j\right\|^2 / 2\sigma^2\right) \tag{5}$$

The width of the RBF kernel depends on the distance between the network centers, which will be explained in detail in the next section. Accordingly, a smoothness loss is defined as the cross-entropy between the smooth label and the classifier output as follows:

$$\mathcal{L}_s(x_i) = -[s_i \ln(f(x_i)) + (1 - s_i)\ln(1 - f(x_i))] \tag{6}$$

Once SSALD triggers the query strategy, an unlabeled instance is selected from W_s according to the assigned probability for labeling. The likelihood of being selected for each instance is as follows:

$$P(\text{label} \mid x_i) = \frac{\mathcal{L}_s(x_i)}{\sum_{j \in W_s} \mathcal{L}_s(x_j)} \tag{7}$$

Instances with higher smoothness loss are likelier to label. During stable periods, the classifier has good accuracy and a similar smoothness loss in most unlabeled instances. This process allows the classifier to focus on the global data distribution, uncovering potential changes. When a change occurs, the smoothness loss in drift regions noticeably increases. Labeling these instances enables quick adaptation to new concepts. This approach helps the model avoid over-focusing on the decision boundary and missing potential changes beyond it, while maintaining the flexibility to promptly adapt to any novel decision boundary as soon as it emerges.

The pseudocode of smoothness loss-based query strategy is shown in Algorithm 1.

Algorithm 1. Smoothness Loss-based Query Strategy.

Input: Sliding window: W_s; Label window: W_l;
Output: Updated sliding window: W_s; Updated label window: W_l

```
W_s^u ← {d_i = {x_i} | d_i ∈ W_s ∧ d_i. is unlabeled}
for d_i ∈ W_s^u do
  s_i ← Calculate smoothness label via Eq. (4)
  L_s(d_i) ← Calculate smoothness loss via Eq. (6)
end
for d_i ∈ W_s^u do
  P(.label | x_i) ← Calculate labeling probability via Eq. (7)
end
Select a instance d_i based on sampling probability
d_i ← d_i ∪ y_i, Label instances by human expert.
W_l ← W_l ∪ d_i, Update label window
if |W_l| > L then
  Remove the oldest instance from W_l
end
```

C. Adaptive Node Adjustment

Usually, RBFNNs use a distance-based similarity measure to calculate the activations of their hidden layer neurons, and the location of RBF centers can have a significant impact on prediction accuracy. Instances far from the RBF centers will have low hidden layer activations, leading to poor prediction accuracy. Thus, to capture important features of the data, the network center should cover the relevant region of the input space. However, concept drift can cause RBF centers to become outdated and less representative of the current data distribution. Soares et al. [48] use learning vector quantization to update all RBF centers on each data chunk, indicating that the output weight of each node must be retrained, which may lose valuable historical information and reduce learning efficiency. Thus, an adaptive node adjustment method is proposed to capture the new data distribution while preserving discriminative information in non-drifted regions.

To evaluate the relevance of each node concerning the current distribution, the representativeness of an RBF center c_i is defined as follows:

$$R(c_i) = 1 - \max_{j \in C, j \neq i} \exp\left(-\frac{\left\|c_i - c_j\right\|^2}{2\sigma^2}\right) + \sum_{j \in W_s} \exp\left(-\frac{\left\|c_i - x_j\right\|^2}{2\sigma^2}\right) \tag{8}$$

In the above equation, the first item reflects the similarity between c_i and other RBF centers. The second item is the contribution of c_i to classification, where $C = \{c_1, \ldots, c_H\}$ is the set of all RBF centers. If the representation of c_i is lower than γ, it will be substituted with the instance x_j that has the highest representation.

$$c_i \leftarrow \arg\max_{j \in W_s} R(x_j) \tag{9}$$

Adaptive node adjustment maximizes the distance between different centers while moving the centers toward the location of a higher density of instances, which follows the data distribution and ensures the diversity of the centers. The width σ is set as follows to prevent the radial basis functions from being overly sharp or overly flat.

$$\sigma = \frac{\max_{i,j \in C} \left\|c_i - c_j\right\|}{\sqrt{2H}} \tag{10}$$

At each time step, SSALD checks the representations of all RBF centers. Once $\exists i, R(c_i) < \gamma$, the center with the lowest

Authorized licensed use limited to: GITAM University. Downloaded on September 10,2025 at 07:15:08 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2. Adaptive Node Adjustment.

Input: RBF centers: $C = \{c_i\}_{i=1}^{H}$; Sliding window W_s; Threshold γ.
Output: updated RBF centers: $C = \{c_i\}_{i=1}^{H}$.

```
for c_i ∈ C do
  R(c_i) ← Calculate the representation via Eq. (8)
end
if min{R(x_i) | i = 1, ..., H} < γ then
  for x_i ∈ W_s do
    R(x_i) ← Calculate the representation via Eq. (8)
  end
end
while min{R(x_i) | i = 1, ..., H} < γ do
  Remove c_i = arg min_{i∈C} R(x_i) from C
  x_i = arg max_{j∈W_s} R(x_j)
  C ← C ∪ x_i
end
Update RBF width σ via Eq. (10)
```

representation is updated and the representations for each center are recomputed. Repeat the above operations until the representativeness of all centers satisfies the condition, and update the width according to (10).

The pseudocode of adaptive node adjustment is shown in Algorithm 2.

D. Semi-Supervised Regularization

After determining the RBF centers, SSALD updates the output weights based on the sliding and label windows. To ensure that the network's output remains consistent across consecutive instances, a semi-supervised regularization term is incorporated into the loss function. This prompts the network to learn decision boundaries for low-density regions, leading to better generalization. The semi-supervised regularization term is defined as the average value of the smoothness loss over unlabeled instances within a sliding window, and cross-entropy is used as the supervised loss for labeled instances.

$$\mathcal{L}(W_s, W_l) = \frac{1}{L+u}\sum_{i\in W_l}\mathcal{L}_l(x_i) + \frac{1}{L+u}\sum_{i\in W_s^u}\mathcal{L}_s(x_i) \tag{11}$$

$$\mathcal{L}_l(x_i) = -w_i[y_i\ln(f(x_i)) + (1-y_i)\ln(1-f(x_i))] \tag{12}$$

The output weights are updated by Adam (Adaptive Moment Estimation) update rule. Different from SGD (stochastic gradient descent), Adam adapts the learning rate for each weight based on the first and second moments of the gradients, while the learning rate of SGD is constant and the same for all weights. In the case of SSALD, the optimal learning rate for each node's output weight may vary after node adjustment. The adjusted nodes require a large learning rate to learn the current concept quickly. On the contrary, other nodes have accumulated much knowledge about the current distribution, and a lower learning rate is more conducive to network convergence. The updating rule for ω_i is given as follows:

$$\begin{aligned}
\omega_i^{(t)} &\leftarrow \omega_i^{(t-1)} - \eta\widehat{m}_i^{(t)}/\sqrt{\hat{v}_i^{(t)}}\\
\widehat{m}_i^{(t)} &= \frac{m_i^{(t)}}{1-\beta_1^t}\\
\hat{v}_i^{(t)} &= \frac{v_i^{(t)}}{1-\beta_2^t}\\
m_i^{(t)} &= \beta_1 m_i^{(t-1)} + (1-\beta_1)\nabla_{\omega_i}\mathcal{L}\\
v_i^{(t)} &= \beta_2 v_i^{(t-1)} + (1-\beta_2)\nabla^2_{\omega_i}\mathcal{L}
\end{aligned} \tag{13}$$

where $m_i^{(t)}$ and $v_i^{(t)}$ are the first and the second moments of the gradient on ω_i, and η is the learning rate. The decay rates β_1 and β_2 control the exponential decay of the moving averages of the gradient and its square, respectively. Let $\Delta\mathcal{L} = \mathcal{L}^{(t)} - \mathcal{L}^{(t-1)}$. There are two stopping conditions, one is $\Delta\mathcal{L} < 0.001$ and the other is reaching the maximum number of training epochs E_{max}.

SSALD is initialized over a sliding window formed by the first N instances of the data stream, where H random instances are assigned as the RBF centers of the network. Additionally, $N * B$ instances are randomly labeled as initial training data and put into the label window. The initialization process of SSALD is shown in Algorithm 3.

Algorithm 3. SSALD Initialization.

Input: Data stream: S; Number of RBF centers: H; Threshold γ; Sliding window size N; label Window size: L; Label budget: B; The maximum number of training epochs: $E_{\max}$.
Output: Initialized SSALD.

```
W_s ← {d_t | d_t ∈ S, t = 1, ..., N}
C ← Randomly select H instances from W_l as RBF centers.
σ ← Update width via Eq. (10)
Randomly label B * N instances from W_s.
W_l ← {d_t | d_t ∈ W_s ∧ d_t. is labeled }
Update network weights via Eq. (13)
epoch = 1
while ΔL < 0.001 and epoch < E_max end
  Update network weights via Eq. (13)
  epoch = epoch + 1
end
```

In general, SSALD performs SSL based on the smoothness assumptions to create smooth decision boundaries, while the smoothness loss-based query strategy selects instances with high smoothness loss to better estimate the smoothness labels for unlabeled data. This interactive learning method realizes the mutual enhancement of SSL and AL. In addition, adaptive node adjustment enables the network with the flexible ability to adapt to concept drift. It can preserve valuable information from historical concepts, thus achieving fast recovery from drift, especially with the scarcity of labels.

Authorized licensed use limited to: GITAM University. Downloaded on September 10,2025 at 07:15:08 UTC from IEEE Xplore. Restrictions apply.

TABLE I Characteristics of the synthetic and real-world datasets.

TYPE	DATASET	FEATURES	NUMBER OF INSTANCES	CLASS SIZE	NOISE RATIO	NUMBER OF DRIFT	DRIFT TYPE
Synthetic (group A)	$agrawal_A$	40	10000	(5001, 4999)	5%	2	S
	$sine_A$	4	10000	(5001, 4999)	None	2	S
	rbf_A	10	10000	(5309, 4691)	None	2	G
	$tree_A$	8	10000	(5338, 4662)	None	2	G
	hyp_A	10	10000	(4980, 5020)	5%	-	I
Synthetic (group B)	$agrawal_B$	40	100000	(50002, 49998)	5%	3	S
	$sine_B$	4	100000	(50002, 49998)	None	3	S
	rbf_B	10	100000	(62269, 37731)	None	3	G
	$tree_B$	8	100000	(57990, 42010)	None	3	G
	hyp_B	10	100000	(50077, 49923)	5%	-	I
	hybrid	10	100000	(50011, 49989)	5%	5	S/G/I
Real	elec	6	45312	(26075, 19237)	unknow	unknow	unknow
	weather	8	18159	(12461, 5689)	unknow	unknow	unknow
	covtype35	54	45247	(35754, 9493)	unknow	unknow	unknow
	covtype36	54	53121	(35754, 17367)	unknow	unknow	unknow
	drebin	300	129013	(123453, 5560)	unknow	unknow	unknow
	jiaolong	14	64186	(56949, 7237)	unknow	unknow	unknow

[1] S: sudden drift; G: gradual drift; I: incremental drift.

IV. Experimental Design

In this paper, four groups of experiments are conducted to verify the effectiveness of the proposed SSALD. First, the sensitivity of the main parameters on algorithm performance is analyzed. Second, the classification accuracy and execution time of proposed method are compared with two AL methods, three semi-supervised methods and three fully supervised methods under different label budgets. Then, a visualization analysis is performed to show how the proposed node adjustment method works. Finally, the rationality of key components in SSALD is investigated through an ablation experiment. All experiments are realized by scikit-multiflow learning library, a framework for data stream mining in Python, and conducted on macOS Ventura 13.2, 16-GB memory, Apple M1 SOC. Each experiment has been independently run five times [38].

A. Benchmark Datasets

As shown in Table I, 10 synthetic data streams and four real-world ones have been employed in the experiments. It can be observed that most of the datasets are relatively balanced, with four synthetic datasets containing noise. To evaluate the impact of different types of drift on model performance, the synthetic datasets include various types of concept drifts. These synthetic data streams are generated by five different data generators, and are divided into two groups. In group A, each data stream has 10,000 instances. For a data stream with sudden drift in this group, a new concept completely replaces the old one over 10 instances at the 3000th time step and 6000th time step. In a data stream with gradual drift, the concept changes within 1000 instances at the same position. The distance between different drifts in this group is 3000 timestamps. In contrast, each data stream in group B contains 100,000 instances, and drifts occur at the 25,000th, 50,000th, and 75,000th time steps. The difference is that the gradual drift process is extended to 5,000 instances, and the time interval between different drifts is increased to 25,000 timestamps. The incremental drifts in hyp_A and hyp_B continue from the beginning to the end of data stream. In addition, a hybrid data stream is generated in group B, incorporating multiple types of concept drift, including two sudden drifts, two gradual drifts and an incremental drift.

Compared with group A, the data streams in group B provide more instances for training before concept drift occurs, implying the different frequencies of producing a drift. Thus, the above two groups of data streams can provide a relatively comprehensive test for the effectiveness of the classification algorithms in terms of their abilities to adapt to changes with different frequencies.

For the real-world data streams, five commonly-used datasets are selected as the benchmarks, including Electricity[1] [54], Weather[2] [11], Covtype[3] [55], Drebin [56] and JiaoLong [57]. Especially, Covtype was originally a multi-class dataset comprising seven distinct classes. We selected three primary classes and formed two binary data streams from them. The detailed

[1][Online]. Available: https://moa.cms.waikato.ac.nz/datasets/
[2][Online]. Available: https://github.com/scikit-multiflow/streaming-datasets
[3][Online]. Available: https://archive.ics.uci.edu/ml/datasets/Covertype.

Authorized licensed use limited to: GITAM University. Downloaded on September 10,2025 at 07:15:08 UTC from IEEE Xplore. Restrictions apply.

information of data generators and real-world datasets is provided in Section S1 of Supplementary.[4]

B. Comparison Design

To verify the effectiveness of the proposed SSALD algorithm, two AL methods, three SSL methods and three supervised learning methods are employed for comparison. These algorithms are briefly discussed as follows.

1) *Rand (AL) [19]*: As a baseline method, the random query strategy randomly labels incoming instances based on the label budget to train classifiers.
2) *RanVarUn (AL) [19]*: The random variable uncertainty strategy selects instances for labeling based on uncertainty of the model. It adjusts the magnitude of the area to search by a random value, to reveal more hidden knowledge.
3) *SSEA (SSL) [58]*: The semi-supervised ensemble approach trains an ensemble classifier using labeled instances and builds an unsupervised model using unlabeled instances. It builds the final semi-supervised ensemble classifier by maximizing the consensus between the supervised ensemble classifier and unsupervised model.
4) *SmSCluster (SSL) [59]*: Semi-supervised micro-clustering uses a semi-supervised algorithm to create K clusters, and extracts a summary of the data statistics in each cluster as a "micro-cluster". The "micro-clusters" are continuously updated by newly arrived instances to adapt to concept drift.
5) *OSNN (SSL) [48]*: OSNN is also a data stream learning method based on RBFNN. It employs SSL vector quantization to train network centers and adjust network weights based on the manifold assumption.
6) *ARF (SL) [60]*: As one of the most effective data stream classification algorithms, it combines the online resampling method and adaptive operators with random forests to cope with concept drifts in data streams.
7) *DWM (SL) [54]*: In dynamic weighted majority, all base classifiers are integrated by the weighted sum. The weights decay with their timestamp and the discriminative ability of base classifier on the current data chunk.
8) *OBA (SL) [61]*: It is an improvement from the Online Bagging algorithm, with the addition of an ADWIN drift detector.

C. Parameter Settings

The parameter settings in SSALD achieve a balance between classification performance and computational efficiency. Specifically, the sliding window size is set to 200 to enforce the smoothness assumption better, which is the same as the label window. For adaptive node adjustment, the representative threshold of RBF centers is set to 0.5 and the number of hidden layer nodes is set to 20 in all data streams. In addition, following the convention [62], the initial learning rate of the Adam optimizer is set to 0.01 and the decay rates are set to the default values in the pytorch library. During the training process, the maximum number of training epochs per time step is set to 100.

The Hoeffding Adaptive Tree is used as the classifier for two AL methods, and the parameters are the default values in the Scikit-multiflow library. SSEA is a chunk-based approach where the chunk size is set to 500 and the ensemble size is set to 6, with three supervised base classifiers and 3 unsupervised ones. In SmSCluster, K is set to 20, chunk size is set to 500 and the number of base classifiers is 10. The remaining parameters are consistent with the default values in the paper. However, the original paper does not give the default parameters of OSNN, and the experiment parameters are randomly sampled. For a fair comparison, the number of hidden layer nodes is set to 20, which is consistent with SSALD. Two trade-off values are set to 0.5, and the width proportions are set to 1.0. ARF, DWM, and OBA are all implemented in the Scikit-multiflow library, so their default parameter values are used.

D. Evaluation Metrics

A common scheme for evaluating data stream algorithms is prequential evaluation. The classifier is tested before being updated with each newly arrived instance. This method enables maximum data utilization, ensures smooth accuracy, and does not require additional storage to preserve the test set. At each time step, the prediction result of the classifier for the current instance can be expressed as follows:

$$L(y_t, \hat{y}_t) = \mathbb{I}(y_t, \hat{y}_t) \tag{14}$$

where $\hat{y}_t$ is the prediction of classifier and $\mathbb{I}(\cdot)$ is an indicator function that outputs 1 when the condition is true; otherwise, it returns 0. The performance of the classifier at time t is the cumulative accuracy of all previous instances:

$$\text{Accuracy} = \frac{1}{t}\sum_{i=1}^{t} L(y_i, \hat{y}_i) \tag{15}$$

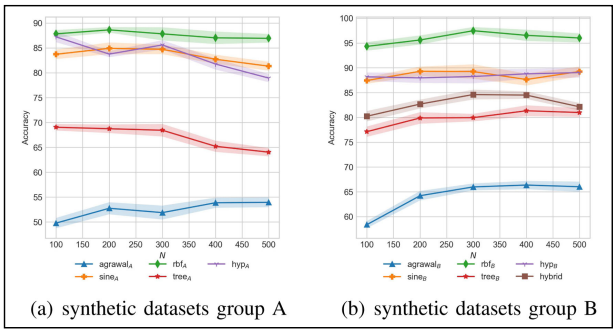
The Friedman test and its posthoc test after Conover are used to compare the classification accuracy of SSALD with other methods. Given k comparison algorithms and n datasets, the statistic of the Friedman test is calculated according to the following equation:

$$\chi_R^2 = \left[\frac{12}{nk(k+1)}\sum_{i=1}^{k} R_i\right] - 3n(k+1) \tag{16}$$

where R_i is the rank of ith algorithm. If the Friedman test rejects the null hypothesis, the Conover post-hoc test can be employed to conduct pairwise comparisons. A significant

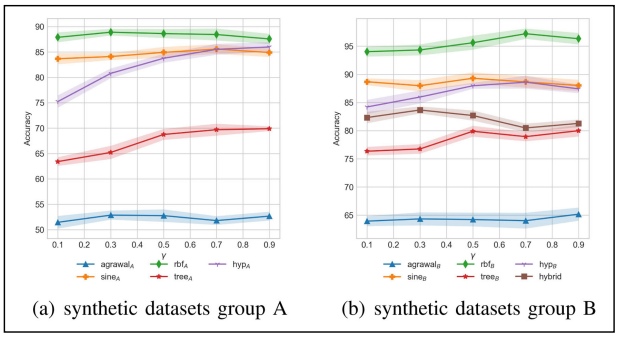
[4][Online]. Available: https://github.com/Jesen-BT/SSALD

Authorized licensed use limited to: GITAM University. Downloaded on September 10,2025 at 07:15:08 UTC from IEEE Xplore. Restrictions apply.



(a) synthetic datasets group A (b) synthetic datasets group B

FIGURE 2 Accuracy of SSALD with different values of N.



(a) synthetic datasets group A (b) synthetic datasets group B

FIGURE 3 Accuracy of SSALD with different values of γ.

difference between the two algorithms exists if the following inequality holds:

$$\left|R_i - R_j\right| > t_{1-\frac{\alpha}{2};(n-1)(k-1)} \times \sqrt{\frac{2k\left(1-\frac{\chi_R^2}{n(k-1)}\right)\left(\sum_{i=1}^{n}\sum_{j=1}^{k} R_{i,j}^2 - \frac{nk(k+1)^2}{4}\right)}{(k-1)(n-1)}} \tag{17}$$

where t is t-distribution. More details about the Friedman test and its post-hoc test can be found in [63].

V. Results and Discussions

A. Parameter Sensitivity Analysis

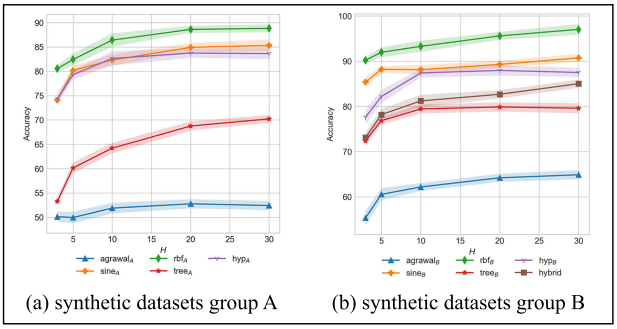
In the proposed SSALD algorithm, we introduce two new parameters, including the size of the sliding window N, and the threshold of RBF center representation γ. To analyze their influence on the classification performance, a group of experiments for SSALD under various values of these parameters have been carried out. Here, $N \in \{100, 200, 300, 400, 500\}$ and $\gamma \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$. In addition, as a key parameter of RBFNN, the number of hidden layer nodes H is also evaluated within $\{3, 5, 10, 20, 30\}$. When a parameter is evaluated, the other parameters remain at the same value listed in Section IV-C, and the label budget is set to $B = 0.05$.

Figure 2 depicts the classification performance of SSALD under different values of N on synthetic stream group A and group B. For synthetic data streams with high frequency of drift, a large window size does not bring significant performance improvement, and even leads to performance degradation in some data streams. A smaller window size helps the model adapt to rapid changes of the data distribution, especially for gradual and incremental drift. In contrast, SSALD with a larger window size achieves better performance on the synthetic stream group B. More training instances at each time step can improve the performance of the proposed algorithm in the periods of stability. For γ, it determines the sensitivity of adaptive node adjustment to distribution changes. From Figure 3, it can be seen that both large and small values of γ lead to a decrease in classification performance. A small value of γ makes the model unable to track distribution changes in time, while a large value of γ leads to frequent updates of the model structure. In both cases, the model cannot accurately learn the current joint probability distribution of the data stream. Different from the previous two parameters, in Figure 4, the classification performances on all data streams become better, as H increases from 3 to 30. However, too many hidden layer nodes may lead to redundancy and reduce the learning efficiency of the model. The experimental results of real-world data streams are shown in Figures S1 of Supplementary. Similar to synthetic data streams, the optimal parameter values of N and γ both depend on the specific characteristics of data streams.

B. Comparisons Between SSALD and Other Methods

1) Overall Results

To provide a comprehensive analysis, all methods are evaluated under two different labeling budgets, i.e., $B \in \{0.01, 0.05\}$. The average prequential accuracy of all comparative algorithms is listed in Tables II and III. It is clear that SSALD is superior to its competitors on 9 and 11 data streams under the budget of 0.01 and 0.05, respectively. Especially on synthetic streams with higher drift frequency, the improvement of SSALD is more noticeable. However, for streams generated by the Agrawal generator, SSALD performs relatively poorly compared to other methods, which might be due to the distance metric used for the radial basis function. The raw agrawal data streams contain nominal attributes, so the Euclidean distance may not



(a) synthetic datasets group A (b) synthetic datasets group B

FIGURE 4 Accuracy of SSALD with different values of H.

Authorized licensed use limited to: GITAM University. Downloaded on September 10,2025 at 07:15:08 UTC from IEEE Xplore. Restrictions apply.

TABLE II Prequential classification accuracy (%) of various methods under $B = 0.01$.

	Rand	RanVarUn	ARF	DWM	OBA	SSEA	SmSCluster	OSNN	SSALD
$agrawal_A$	52.45±1.21	53.77±0.47	55.36±0.60	51.88±0.31	51.65±0.38	51.23±2.16	53.93±0.37	52.69±0.90	52.21±1.65
$sine_A$	41.05±1.06	39.53±4.91	43.29±2.62	52.95±0.77	40.26±3.52	54.35±0.32	68.72±0.60	60.32±1.62	65.91±1.36
rbf_A	67.30±1.45	68.34±1.17	77.57±1.04	69.70±0.68	74.46±0.35	64.20±1.89	63.39±1.18	77.08±0.57	81.55±1.31
$tree_A$	57.41±1.72	57.41±2.70	59.65±0.87	62.87±1.18	55.71±1.93	57.96±3.22	60.57±1.86	61.80±0.32	64.83±0.81
hyp_A	85.72±0.57	84.90±0.58	72.53±0.15	79.43±0.25	78.22±1.96	57.77±0.57	62.00±1.39	74.77±0.54	84.42±0.39
$agrawal_B$	58.67±0.98	56.84±0.76	62.99±3.05	55.39±0.51	53.12±0.43	50.18±0.30	53.93±0.45	54.85±0.53	60.55±1.64
$sine_B$	61.70±1.30	65.53±0.80	78.31±0.69	68.82±1.82	64.99±0.23	60.13±0.06	79.51±0.46	83.13±0.69	86.13±1.80
rbf_B	69.09±0.51	71.53±0.54	83.29±0.11	73.29±0.38	89.21±0.06	62.71±0.62	73.93±0.22	81.09±0.06	92.61±1.37
$tree_B$	56.73±0.27	57.72±0.66	67.68±1.87	62.71±0.62	65.55±1.32	62.03±1.92	68.57±0.34	70.71±0.03	73.89±1.30
hyp_B	84.57±0.15	84.37±0.50	77.53±0.19	86.67±0.31	80.80±1.85	50.90±0.11	70.88±0.29	79.91±0.93	85.83±1.05
hybrid	78.55±1.28	75.64±2.41	72.33±0.88	78.03±0.52	75.02±3.44	56.33±0.86	64.39±0.56	65.33±1.47	81.39±0.95
elec	73.91±1.21	72.83±0.59	71.22±0.95	68.31±0.89	63.85±0.39	73.88±0.40	62.95±0.55	72.43±0.81	72.70±0.14
weather	69.99±2.19	69.21±1.08	72.44±0.29	68.10±0.18	74.09±1.49	68.51±1.01	69.06±0.66	71.57±1.12	74.94±2.61
covtype35	92.60±2.09	92.35±0.97	93.69±0.05	92.99±1.30	95.44±0.87	88.98±2.47	88.62±0.95	90.18±0.98	93.06±1.99
covtype36	55.88±3.44	58.34±1.29	76.51±0.47	64.52±0.59	74.77±0.76	67.63±0.57	74.33±0.45	79.26±1.14	80.17±0.33
drebin	95.33±0.29	94.18±0.14	96.46±0.42	92.10±0.37	95.44±0.31	95.38±0.15	95.40±0.05	97.58±0.11	96.28±0.19
jiaolong	84.21±1.44	85.49±0.22	89.76±0.82	86.79±1.49	88.56±1.19	80.44±1.27	84.01±0.45	90.53±2.01	92.39±0.91
rank	5.53(6)	5.76(7)	3.82(2)	5.29(5)	5.24(4)	7.47(9)	5.94(8)	3.94(3)	2.00(1)

accurately reflect the relationship between different instances. In addition, a Friedman test was also conducted to determine whether the methods differed significantly in classification accuracy. Tables IV and V show that the null hypothesis is rejected, followed by the post-hoc Conover test. It can be noted that SSALD outperforms all comparative algorithms except ARF, with a significance level of $p < 0.05$. With an increase in the label budget, there is usually an improvement in the performance of all algorithms, and SSALD consistently maintains its top ranking among them.

TABLE III Prequential classification accuracy (%) of various methods under $B = 0.05$.

	Rand	RanVarUn	ARF	DWM	OBA	SSEA	SmSCluster	OSNN	SSALD
$agrawal_A$	52.52±0.15	52.91±0.70	63.47±1.26	53.14±0.00	52.22±0.25	54.97±2.91	56.39±0.13	54.00±1.00	52.76±0.36
$sine_A$	50.31±1.13	46.56±5.10	78.02±3.15	78.70±0.95	64.56±2.78	60.66±1.22	78.98±0.40	80.66±1.32	84.43±1.37
rbf_A	76.80±1.72	79.04±0.13	85.31±0.66	76.81±1.36	88.27±1.28	75.26±1.32	85.57±1.25	81.66±0.69	88.47±0.47
$tree_A$	58.24±1.39	64.07±2.08	67.81±1.68	58.33±0.40	59.70±2.41	67.12±2.38	64.87±1.02	64.03±0.22	68.10±0.87
hyp_A	84.03±0.54	87.02±0.12	74.57±0.05	85.26±0.45	80.99±1.61	67.27±0.20	72.99±1.23	75.49±0.36	83.10±0.47
$agrawal_B$	62.78±0.43	65.53±0.35	71.00±2.97	56.65±0.05	59.78±0.25	50.45±0.22	57.08±0.21	55.20±0.03	64.22±1.30
$sine_B$	81.96±1.26	80.91±1.06	87.73±0.28	83.05±0.93	86.21±0.14	66.03±0.65	86.59±0.11	84.56±0.55	89.30±1.74
rbf_B	78.44±0.01	79.41±0.12	89.16±0.06	81.00±0.78	94.37±0.26	66.40±0.44	91.84±0.29	82.87±0.13	95.61±1.65
$tree_B$	67.33±0.14	69.95±0.03	79.79±1.76	74.59±0.64	73.98±0.72	66.31±1.54	74.53±0.18	72.05±0.05	79.89±0.85
hyp_B	86.27±0.08	86.10±0.31	80.97±0.26	88.70±0.10	81.91±0.46	52.80±0.18	78.71±0.31	81.40±0.19	87.97±1.41
hybrid	83.33±0.59	77.20±0.60	76.31±1.35	81.20±1.06	76.98±0.79	63.18±0.41	70.20±0.46	72.15±1.25	82.54±0.84
elec	73.26±0.40	74.54±0.55	77.48±0.84	72.84±1.20	67.96±0.34	72.91±0.47	67.39±1.03	74.14±0.62	79.40±0.97
weather	71.89±2.32	69.19±1.41	73.78±0.45	65.54±0.16	74.73±1.48	68.68±0.91	73.63±1.28	71.69±0.70	75.29±1.66
covtype35	91.74±1.60	88.88±0.05	96.21±0.44	94.52±0.89	95.61±0.62	92.16±2.71	93.13±0.64	91.77±0.89	93.89±1.32
covtype36	65.70±3.22	72.17±0.53	82.29±0.52	76.23±1.04	78.83±0.32	69.53±0.26	77.36±0.63	81.02±0.63	83.68±0.99
drebin	96.83±0.15	96.02±0.22	97.44±0.27	95.39±0.33	97.01±0.58	96.70±0.11	95.21±0.71	95.99±0.41	97.68±0.25
jiaolong	89.54±2.02	88.36±1.05	93.20±0.77	90.49±0.88	92.71±0.94	86.57±0.72	87.46±0.55	94.88±1.64	96.62±1.41
rank	6.12(8)	5.76(7)	3.18(2)	5.29(5)	4.59(3)	7.47(9)	5.41(6)	5.24(4)	1.94(1)

Authorized licensed use limited to: GITAM University. Downloaded on September 10,2025 at 07:15:08 UTC from IEEE Xplore. Restrictions apply.

TABLE IV Friedman test and its post-hoc test of all the methods over classification accuracy under $B = 0.01$.

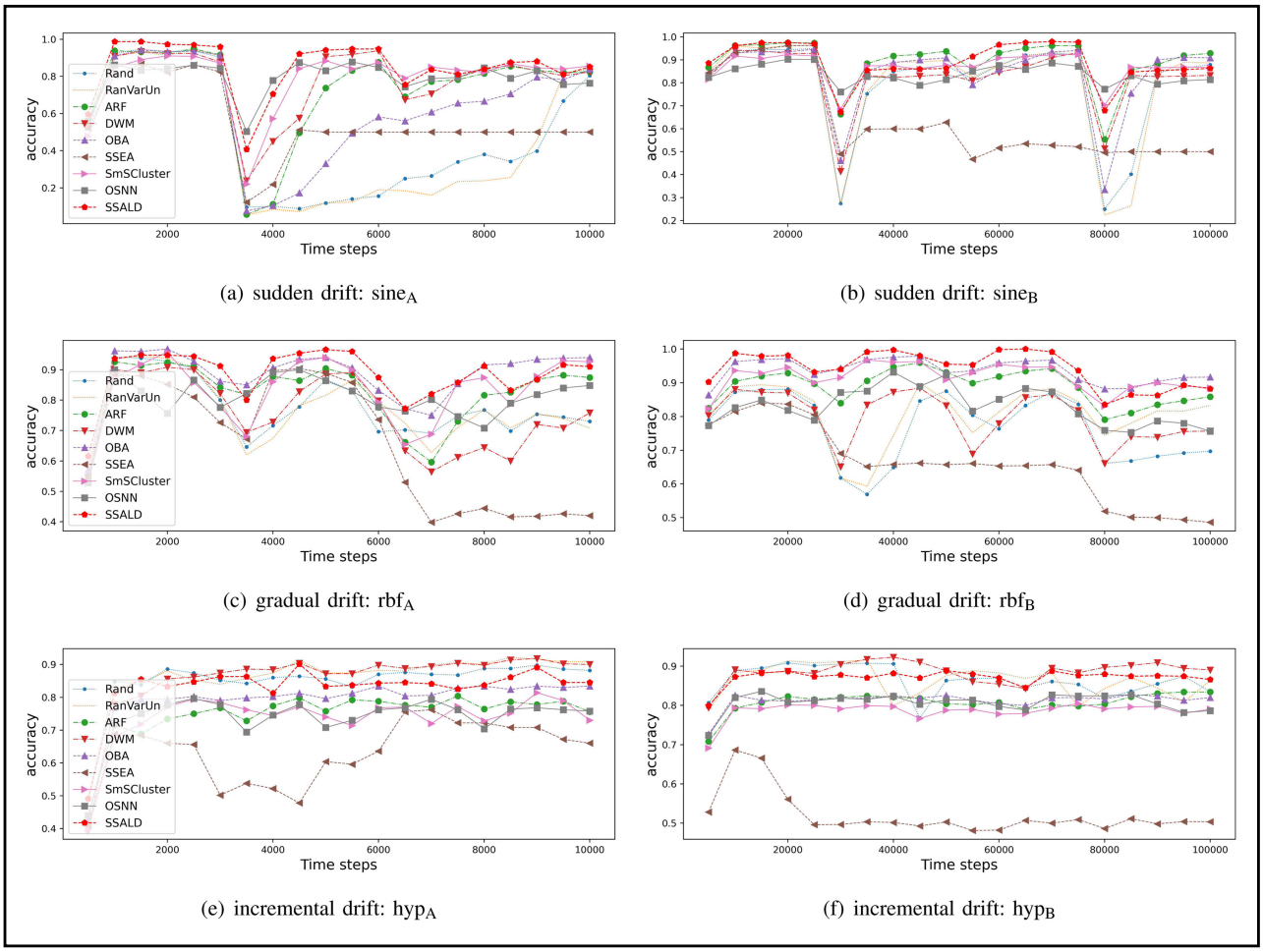
FRIEDMAN TEST	$\mathcal{X}_R^2$			P-VALUE OF $\mathcal{X}_R^2$				df
	44.20			$5.20e^{-7}$				8
Post-hoc test for SSALD	Rand	RanVarUn	ARF	DWM	OBA	SSEA	SmSCluster	OSNN
P-value	$3.41e^{-4}$	$1.40e^{-4}$	0.059	$7.97e^{-4}$	$9.79e^{-4}$	$7.62e^{-8}$	$6.99e^{-5}$	0.045

TABLE V Friedman test and its post-hoc test of all the methods over classification accuracy under $B = 0.05$.

FRIEDMAN TEST	$\mathcal{X}_R^2$			P-VALUE OF $\mathcal{X}_R^2$				df
	47.83			$1.07e^{-7}$				8
Post-hoc test for SSALD	Rand	RanVarUn	ARF	DWM	OBA	SSEA	SmSCluster	OSNN
P-value	$2.69e^{-5}$	$1.11e^{-4}$	0.199	$6.47e^{-4}$	$6.61e^{-3}$	$5.72e^{-8}$	$4.23e^{-4}$	$7.97e^{-4}$

2) Performance Curve

To analyze the performance trend of the model in the face of different types of drift, the real-time accuracy curves of different algorithms under $B = 0.05$ are plotted using the chunk-based prequential evaluation method. Each point in the curve is the performance of the classifier on the previous data chunk obtained by prequential evaluation. More especially, the performance curves of each algorithm on sine_A and sine_B with sudden drift are depicted in Figures 5(a) and (b). Apparently, all algorithms show a significant performance drop after a sudden drift. However, SSALD can recover from sudden drifts more rapidly than the other methods. In addition, it can be seen that drift has a more significant effect on the performance of other algorithms. This is because the adaptive node



(a) sudden drift: sine_A (b) sudden drift: sine_B

(c) gradual drift: rbf_A (d) gradual drift: rbf_B

(e) incremental drift: hyp_A (f) incremental drift: hyp_B

FIGURE 5 Prequential accuracy of all comparative classifiers at each time step for data streams with different types of drift.

Authorized licensed use limited to: GITAM University. Downloaded on September 10,2025 at 07:15:08 UTC from IEEE Xplore. Restrictions apply.

The complete learning process on the entire synthetic data stream is uploaded to YouTube, where the behavior of SSALD at each time step can be viewed.

adjustment in SSALD can better retain valuable historical information without re-learning the entire data distribution after drift. In terms of rbf_A and rbf_B, in Figures 5(c) and (d), SSALD also outperformed other methods under gradual drift. In particular, in datasets with a high frequency of drift, SL methods cannot recover from drift due to insufficient instances, reflecting that exploiting information from unlabeled instances can improve classifier performance when labels are scarce. Under incremental drift, the prequential accuracy of all comparators at each time step is depicted in Figures 5(e) and (f). Since drift has always been present, accompanied by numerous intermediate concepts, the classification accuracy of most classifiers tends to fluctuate. Nevertheless, SSALD still maintains relatively stable performance throughout the entire data stream.

3) Execution Time

Under $B = 0.05$, the execution time of all methods on each data stream is listed in Table VI. It can be noted that SSL methods generally take more time because of learning from unlabeled instances. The computation cost of SSALD mainly comes from distance computation in radial basis functions. Thus, the dimensionality has a great influence on the efficiency of the proposed algorithm. Therefore, conducting feature selection [64] before model training can help improve the efficiency of SSALD. In addition, it is possible to speed up the model by specifying a small number of new instances to accumulate before updating the model. However, this might increase the time to recover from drift.

C. Visualization on Tracing Concept Drift

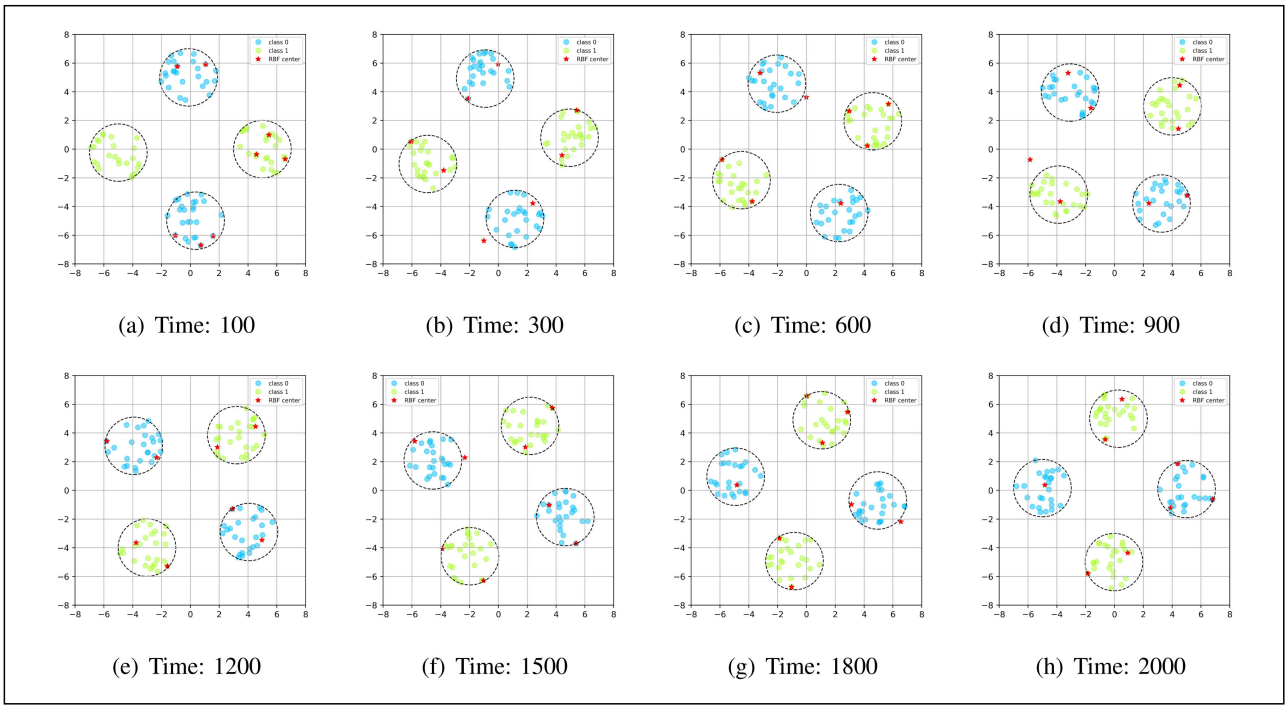
In this experiment, we analyze the rationality of the proposed adaptive node adjustment by visualizing the data distribution and RBF centers variation on a two-dimensional synthetic data stream.

The instances of the synthetic data stream are evenly generated in four circles with [0, 5], [5, 0], [-5, 0] and [0, -5] as the initial centers, and the radius of each circle is 2. To simulate concept drift, the centers of these circles move at a constant speed along an arc, and 2000 instances are generated. By setting the sliding window size of SSALD to 100, and the number of hidden layer nodes to 8, Figure 6 presents some consecutive plots of RBF center adjustment performed by SSALD. Figure 6(a) shows the initial learning stage of SSALD. In the two-dimensional feature space, there are two classes of instances, identified by green and blue, respectively. Each class consists of two micro-clusters with their data distribution bounded by circles. It can be seen that although the initial RBF centers of SSALD do not cover all data distribution regions, they are subsequently adjusted to more representative instances in the data space through the adaptive node adjustment. In addition, as shown in Figures 6(b)–(h), it is evident

TABLE VI Execution time (S) on all datasets under $B = 0.05$.

	Rand	RanVarUn	ARF	DWM	OBA	SSEA	SmSCluster	OSNN	SSALD
$agrawal_A$	1.42	1.05	5.74	3.66	15.19	376.30	245.14	238.65	593.81
$sine_A$	0.38	0.44	3.21	0.69	7.05	224.63	235.98	139.78	187.19
rbf_A	0.52	0.59	3.96	1.26	7.07	315.39	223.20	173.95	119.49
$tree_A$	0.44	0.51	3.74	0.68	7.08	320.43	218.50	155.36	285.35
hyp_A	0.56	0.62	3.96	0.87	8.58	363.33	222.39	171.77	221.43
$agrawal_B$	8.68	10.37	58.69	52.46	322.63	4015.76	3486.76	2075.55	5823.52
$sine_B$	4.60	5.15	34.16	8.35	109.92	2768.00	2576.11	910.45	1947.34
rbf_B	5.51	6.38	40.02	17.00	180.87	2943.52	2854.58	1147.33	2012.22
$tree_B$	5.32	5.58	42.03	16.04	165.76	3413.49	3655.53	1410.80	3211.45
hyp_B	7.56	7.88	53.83	18.69	249.09	5170.44	2680.46	1132.59	3981.21
hybrid	9.17	6.90	49.21	28.40	298.66	4820.05	2610.77	1059.62	4625.29
elec	1.81	2.13	14.56	5.11	74.95	1171.80	1130.45	473.24	655.40
weather	0.69	1.03	8.05	2.66	26.53	464.42	428.25	181.01	488.96
covtype35	5.86	6.51	23.45	13.60	129.06	1636.82	1508.19	1395.95	3199.45
covtype36	6.04	5.74	32.53	30.04	176.42	2073.70	1789.01	1791.10	4098.21
drebin	29.37	32.77	180.37	175.28	969.20	5924.33	6005.32	6924.06	9530.41
jiaolong	9.38	7.29	41.22	39.10	293.40	2857.22	3098.15	2688.47	2955.01

Authorized licensed use limited to: GITAM University. Downloaded on September 10,2025 at 07:15:08 UTC from IEEE Xplore. Restrictions apply.



(a) Time: 100 (b) Time: 300 (c) Time: 600 (d) Time: 900

(e) Time: 1200 (f) Time: 1500 (g) Time: 1800 (h) Time: 2000

FIGURE 6 Visualization of RBF centers and synthetic dataset at different time steps.

TABLE VII Classification accuracy (%) of ssald and its variants.

	SSALD	$\text{SSALD}_{\text{NoSSL}}$	$\text{SSALD}_{\text{NoAL}}$	$\text{SSALD}_{\text{NoNA}}$	$\text{SSALD}_{\text{Base}}$
agrawal_A	52.76±0.62	51.23±1.54	51.29±1.47	52.05±1.32	50.87±0.78
sine_A	84.43±0.34	77.31±0.50	84.45±0.44	82.42±1.16	71.11±0.83
rbf_A	88.47±1.61	83.22±0.73	89.04±0.27	86.41±1.06	74.55±2.09
tree_A	68.10±1.27	57.40±1.11	66.51±1.37	58.05±1.45	57.32±1.67
hyp_A	83.10±1.19	77.65±0.82	82.51±0.86	79.12±0.55	70.15±3.11
agrawal_B	64.22±1.30	58.14±1.06	63.99±0.93	62.78±1.32	55.13±1.40
sine_B	89.30±0.65	82.44±1.18	87.98±2.53	89.76±0.24	83.15±1.58
rbf_B	95.61±0.34	89.14±1.06	96.42±1.52	92.37±1.48	87.51±1.29
tree_B	79.89±0.17	75.02±0.44	76.24±1.72	75.32±1.13	74.47±1.63
hyp_B	87.97±1.53	82.97±0.80	88.65±1.90	74.12±1.23	75.32±1.91
hybrid	82.54±0.84	73.19±0.99	83.12±2.08	74.19±1.58	70.98±1.08
elec	79.40±1.12	69.12±0.70	78.14±1.37	77.41±1.91	71.25±1.35
weather	75.29±0.74	61.43±1.17	74.03±2.01	72.10±2.60	59.55±2.13
covtype35	93.89±0.65	84.46±0.48	89.24±1.85	89.01±0.40	79.18±1.47
covtype36	83.68±1.51	75.12±0.49	83.10±1.40	80.20±0.99	73.40±2.11
drebin	97.68±0.25	95.33±0.41	97.31±0.28	97.10±0.19	95.24±0.54
jiaolong	96.62±1.41	91.38±0.60	93.33±1.72	93.09±0.74	88.51± 1.65

that the RBF centers can continually track changes in the data distribution, and the adjustment mechanism, which takes into account diversity, prevents redundancy of RBF centers. This provides the basis for SSALD to learn good classification boundaries and maintain a good structure in a dynamic environment.

The complete learning process on the entire synthetic data stream is uploaded to YouTube,[5] where the behavior of SSALD at each time step can be viewed.

[5][Online]. Available: https://www.youtube.com/watch?v=tDlatFv3YYg

Authorized licensed use limited to: GITAM University. Downloaded on September 10,2025 at 07:15:08 UTC from IEEE Xplore. Restrictions apply.

Based on the complexity of the data, dynamically adjusting the number of nodes may further improve the efficiency of the algorithm while ensuring the performance of the model.

D. Ablation Study

To verify the rationality of the key components and analyze their impact on the overall classification performance, four variants of SSALD were designed using an ablation method as follows.

$\text{SSALD}_{\text{NoSSL}}$: Without semi-supervised regularization, only the instances labeled by the query strategy are used to update the network.

$\text{SSALD}_{\text{NoAL}}$: The proposed smoothness loss-based query strategy is replaced by a random strategy.

$\text{SSALD}_{\text{NoNA}}$: The RBF centers of hidden layer nodes are fixed and will not be adjusted as the distribution changes. This is achieved by setting $\gamma = 0$.

$\text{SSALD}_{\text{Base}}$: As a baseline, instances are labeled using a random strategy without semi-supervised regularization and no adaptive node adjustment.

The classification accuracy of the above methods under label budget $B = 0.05$ is listed in Table VII. Particularly, due to the absence of semi-supervised regularization, the classification performance of $\text{SSALD}_{\text{NoSSL}}$ is significantly worse than SSALD on all data streams, proving that utilizing knowledge of unlabeled instances can promote performance improvement. $\text{SSALD}_{\text{NoAL}}$ achieves slightly higher accuracy than SSALD on some datasets and has less difference in their overall performance. However, it exhibits a more significant standard deviation than SSALD on most datasets, indicating the weak stability on performance due to the inherent randomness of the random strategy. This also reflects that SSL depends to some extent on the quality of labeled instances, so combining AL and SSL is a feasible solution. $\text{SSALD}_{\text{NoNA}}$ produces poor metrics on most data streams due to the lack of adaptive node adjustment. The reason is that the RBF centers affect the capacity and generalization ability of the network, and a poor network structure will lead to low prediction accuracy. Finally, as a baseline, $\text{SSALD}_{\text{Base}}$ is worse than all other variants. In summary, each component of SSALD is effective and necessary for performance improvement, and all together can achieve the best performance.

VI. Conclusion

This paper proposed a semi-supervised active learning neural network to leverage a small number of labeled instances and abundant unlabeled instances to improve the classification accuracy on data streams with concept drift. To extract knowledge from unlabeled data, a semi-supervised regularization is designed to maximize the output smoothness of the network. The ablation study showed that the semi-supervised regularization builds more accurate decision boundaries and can promote the model to recover quickly from a drift. Then, a smoothing loss-based query strategy is proposed. It further improves the performance of SSALD by selecting more valuable instances and propagating their label information more effectively. Based on visualization experiments, the proposed adaptive node adjustment can track distribution changes over time so that the network maintains a good structure. The comparative experiment results showed that the proposed SSALD method could achieve better classification accuracy than a series of existing AL, SSL and fully supervised learning methods.

Future work should explore the extension of SSALD to utilize its full potential for multi-classification problems in data streams. In addition, the number of hidden layer nodes in SSALD is fixed. Based on the complexity of the data, dynamically adjusting the number of nodes may further improve the efficiency of the algorithm while ensuring the performance of the model.

Acknowledgment

This work was supported in part by the National Natural Science Foundation of China under Grant U23A20340, Grant 61973305, and Grant 52121003, in part by the National Key R&D Program of China under Grant 2022YFB4703700, in part by the Open Funding of State Key Laboratory of Intelligent Coal Mining and Strata Control under Grant SKLIS202403, in part by the Open Foundation of State Key Laboratory for Mining Response and Disaster Prevention and Control of Deep Coal Mines under Grant SKLMRDPC22KF21, in part by the Marsden Fund of New Zealand Government under Contract VUW1913, Contract VUW1914 and Contract VUW2213, in part by the MBIE Data Science SSIF Fund under Contract RTVU1914, and in part by the Fundamental Research Funds for the Central Universities under Contract 24CX06071A.

References

[1] Z. Tong, W. Li, B. Zhang, F. Jiang, and G. Zhou, "Online bearing fault diagnosis based on a novel multiple data streams transmission scheme," *IEEE Access*, vol. 7, pp. 66644–66654, 2019.

[2] A. Preece et al., "Sentinel: A codesigned platform for semantic enrichment of social media streams," *IEEE Trans. Comput. Social Syst.*, vol. 5, no. 1, pp. 118–131, Mar. 2018.

[3] G. Ditzler and R. Polikar, "Incremental learning of concept drift from streaming imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 25, no. 10, pp. 2283–2301, Oct. 2013.

[4] B. Jiao, Y. Guo, C. Yang, J. Pu, Z. Zheng, and D. Gong, "Incremental weighted ensemble for data streams with concept drift," *IEEE Trans. Artif. Intell.*, vol. 5, no. 1, pp. 92–103, Jan. 2024.

[5] Y. Song, J. Lu, A. Liu, H. Lu, and G. Zhang, "A segment-based drift adaptation method for data streams," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 9, pp. 4876–4889, Sep. 2022.

[6] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Proc. 17th Braz. Symp. Artif. Intell.*, 2004, vol. 8, pp. 286–295.

Authorized licensed use limited to: GITAM University. Downloaded on September 10,2025 at 07:15:08 UTC from IEEE Xplore. Restrictions apply.

[7] H. M. Gomes, J. Read, A. Bifet, J. P. Barddal, and J. Gama, "Machine learning for streaming data: State of the art, challenges, and opportunities," *ACM SIGKDD Explorations Newslett.*, vol. 21, no. 2, pp. 6–22, Nov. 2019.
[8] J. D. C. Júnior, E. R. Faria, J. A. Silva, J. Gama, and R. Cerri, "Novelty detection for multi-label stream classification under extreme verification latency," *Appl. Soft Comput.*, vol. 141, 2023, Art. no. 110265.
[9] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, Dec. 2019.
[10] H. Yu, W. Liu, J. Lu, Y. Wen, X. Luo, and G. Zhang, "Detecting group concept drift from multiple data streams," *Pattern Recognit.*, vol. 134, 2023, Art. no. 109113.
[11] A. Liu, J. Lu, and G. Zhang, "Diverse instance-weighting ensemble based on region drift disagreement for concept drift adaptation," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 293–307, Jan. 2021.
[12] J. Gama and G. Castillo, "Learning with local drift detection," in *Proc. 2nd Int. Conf. Adv. Data Mining Appl.*, 2006, vol. 4093, pp. 42–55.
[13] Y. Lu, Y.-M. Cheung, and Y. Tang, "Dynamic weighted majority for incremental learning of imbalanced data streams with concept drift," in *Proc. 26th Int. Joint Conf. Artif. Intell.*, 2017, pp. 2393–2399.
[14] S. G. Soares and R. Araújo, "An on-line weighted ensemble of regressor models to handle concept drifts," *Eng. Appl. Artif. Intell.*, vol. 37, pp. 392–406, 2015.
[15] B. Jiao, Y. Guo, D. Gong, and Q. Chen, "Dynamic ensemble selection for imbalanced data streams with concept drift," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 1, pp. 1278–1291, Jan. 2024.
[16] H. M. Gomes, M. Grzenda, R. Mello, J. Read, M. H. L. Nguyen, and A. Bifet, "A survey on semi-supervised learning for delayed partially labelled data streams," *ACM Comput. Surv.*, vol. 55, no. 4, Nov. 2022, Art. no. 75.
[17] S. Liu et al., "Online active learning for drifting data streams," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 1, pp. 186–200, Jan. 2023.
[18] L. Zhu, S. Pang, A. Sarrafzadeh, T. Ban, and D. Inoue, "Incremental and decremental max-flow for online semi-supervised learning," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 8, pp. 2115–2127, Aug. 2016.
[19] I. Žliobaitė, A. Bifet, B. Pfahringer, and G. Holmes, "Active learning with drifting streaming data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 25, no. 1, pp. 27–39, Jan. 2014.
[20] J. Guo et al., "Semi-supervised active learning for semi-supervised models: Exploit adversarial examples with graph-based virtual labels," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2021, pp. 2896–2905.
[21] M. Gao, Z. Zhang, G. Yu, S. O. Arik, L. S. Davis, and T. Pfister, "Consistency-based semi-supervised active learning: Towards minimizing labeling cost," in *Proc. Comput. Vis.–ECCV 2020*, 2020, pp. 510–526.
[22] S. Mittal, M. Tatarchenko, O. Çiçek, and T. Brox, "Parting with illusions about deep active learning," 2019, *arXiv:1912.05361*.
[23] K. Wang, D. Zhang, Y. Li, R. Zhang, and L. Lin, "Cost-effective active learning for deep image classification," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 27, no. 12, pp. 2591–2600, Dec. 2017.
[24] Y. Lei, L. Ding, and W. Zhang, "Generalization performance of radial basis function networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 26, no. 3, pp. 551–564, Mar. 2015.
[25] H. Yu, P. D. Reiner, T. Xie, T. Bartczak, and B. M. Wilamowski, "An incremental design of radial basis function networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 25, no. 10, pp. 1793–1803, Oct. 2014.
[26] L. Herrera, H. Pomares, I. Rojas, A. Guillén, G. Rubio, and J. Urquiza, "Global and local modelling in RBF networks," *Neurocomputing*, vol. 74, no. 16, pp. 2594–2602, 2011.
[27] B. Walczak and D. Massart, "Local modelling with radial basis function networks," *Chemometrics Intell. Lab. Syst.*, vol. 50, no. 2, pp. 179–198, 2000.
[28] L. Zhou, M. Wu, M. Xu, H. Geng, and L. Duan, "Research of data mining approach based on radial basis function neural networks," in *Proc. 2nd Int. Symp. Knowl. Acquisition Model.*, 2009, vol. 2, pp. 57–61.
[29] M. F. A. Hady, F. Schwenker, and G. Palm, "Semi-supervised learning for tree-structured ensembles of RBF networks with co-training," *Neural Netw.*, vol. 23, no. 4, pp. 497–509, 2010.
[30] B. Krawczyk, L. L. Minku, J. Gama, J. Stefanowski, and M. Woniak, "Ensemble learning for data stream analysis: A survey," *Inf. Fusion*, vol. 37, pp. 132–156, 2017.
[31] L. L. Minku and X. Yao, "DDD: A new ensemble approach for dealing with concept drift," *IEEE Trans. Knowl. Data Eng.*, vol. 24, no. 4, pp. 619–633, Apr. 2012.
[32] G. Oliveira, L. L. Minku, and A. L. I. Oliveira, "Tackling virtual and real concept drifts: An adaptive gaussian mixture model approach," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 2, pp. 2048–2060, Feb. 2023.
[33] H. M. Gomes, J. Read, A. Bifet, and R. J. Durrant, "Learning from evolving data streams through ensembles of random patches," *Knowl. Inf. Syst.*, vol. 63, no. 7, pp. 1597–1625, 2021.
[34] A. Shaker and E. Lughofer, "Resolving global and local drifts in data stream regression using evolving rule-based models," in *Proc. 2013 IEEE Conf. Evolving Adaptive Intell. Syst.*, 2013, pp. 9–16.
[35] C. Fahy, S. Yang, and M. Gongora, "Classification in dynamic data streams with a scarcity of labels," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 4, pp. 3512–3524, Apr. 2023.
[36] D. Ienco, A. Bifet, I. Žliobaitė, and B. Pfahringer, "Clustering based active learning for evolving data streams," in *Proc. Discov. Sci.: 16th Int. Conf., DS 2013*, Singapore, 2013, pp. 79–93.
[37] J. Zgraja, J. Gama, and M. Woźniak, "Active learning by clustering for drifted data stream classification," in *Proc. ECML PKDD 2018 Workshops: DMLE 2018 IoT-Stream 2018*. Dublin, Ireland, 2019, pp. 80–90.
[38] H. Zhang, W. Liu, and Q. Liu, "Reinforcement online active learning ensemble for drifting imbalanced data streams," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 8, pp. 3971–3983, Aug. 2022.
[39] R. Alabdulrahman, H. Viktor, and E. Paquet, "An active learning approach for ensemble-based data stream mining," in *Proc. 2016 Int. Conf. Knowl. Discov. Inf. Retrieval*, 2016, vol. 2, pp. 275–282.
[40] B. Krawczyk, "Active and adaptive ensemble learning for online activity recognition from data streams," *Knowl. Based Syst.*, vol. 138, pp. 69–78, 2017.
[41] Y. Guo, Z. Zheng, J. Pu, B. Jiao, D. Gong, and S. Yang, "Robust online active learning with cluster-based local drift detection for unbalanced imperfect data," *Appl. Soft Comput.*, vol. 165, 2024, Art. no. 112051.
[42] S. Mohamad, A. Bouchachia, and M. Sayed-Mouchaweh, "A Bi-criteria active learning algorithm for dynamic data streams," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 1, pp. 74–86, Jan. 2018.
[43] T. Qiu, X. Liu, X. Zhou, W. Qu, Z. Ning, and C. L. P. Chen, "An adaptive social spammer detection model with semi-supervised broad learning," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 10, pp. 4622–4635, Oct. 2022.
[44] J. Tanha, N. Samadi, Y. Abdi, and N. Razzaghi-Asl, "Cpssds: Conformal prediction for semi-supervised classification on data streams," *Inf. Sci.*, vol. 584, pp. 212–234, 2022.
[45] X. Yang, Z. Song, I. King, and Z. Xu, "A survey on deep semi-supervised learning," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 9, pp. 8934–8954, Sep. 2023.
[46] F. Shen, H. Yu, K. Sakurai, and O. Hasegawa, "An incremental online semi-supervised active learning algorithm based on self-organizing incremental neural network," *Neural Comput. Appl.*, vol. 20, pp. 1061–1074, 2011.
[47] P. Zhang, X. Zhu, J. Tan, and L. Guo, "Classifier and cluster ensembles for mining concept drifting data streams," in *Proc. 2010 IEEE Int. Conf. Data Mining*, 2010, pp. 1175–1180.
[48] R. G. F. Soares and L. L. Minku, "OSNN: An online semisupervised neural network for nonstationary data streams," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 9, pp. 6029–6041, Sep. 2023.
[49] X. Gu, "A dual-model semi-supervised self-organizing fuzzy inference system for data stream classification," *Appl. Soft Comput.*, vol. 136, 2023, Art. no. 110053.
[50] H. Zhang and R. Fu, "An ensemble learning–online semi-supervised approach for vehicle behavior recognition," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 8, pp. 10610–10626, Aug. 2022.
[51] C. Fahy, S. Yang, and M. Gongora, "Scarcity of labels in non-stationary data streams: A survey," *ACM Comput. Surv.*, vol. 55, no. 2, pp. 1–39, 2022.
[52] J. E. V. Engelen and H. H. Hoos, "A survey on semi-supervised learning," *Mach. Learn.*, vol. 109, no. 2, pp. 373–440, 2020.
[53] F. Buchert, N. Navab, and S. T. Kim, "Exploiting diversity of unlabeled data for label-efficient semi-supervised active learning," in *Proc. 26th Int. Conf. Pattern Recognit.*, 2022, pp. 2063–2069.
[54] J. Z. Kolter and M. A. Maloof, "Dynamic weighted majority: An ensemble method for drifting concepts," *J. Mach. Learn. Res.*, vol. 8, no. 12, pp. 2755–2790, 2007.
[55] Y. Sun, K. Tang, L. L. Minku, S. Wang, and X. Yao, "Online ensemble learning of data streams with gradually evolved classes," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 6, pp. 1532–1545, Jun. 2016.
[56] F. Ceschin, M. Botacin, H. M. Gomes, F. Pinagá, L. S. Oliveira, and A. Grégio, "Fast & furious: On the modelling of malware detection as an evolving data stream," *Expert Syst. With Appl.*, vol. 212, 2023, Art. no. 118590.
[57] S. Hu, Z. Liu, M. Li, and X. He, "CADM + : Confusion-based learning framework with drift detection and adaptation for real-time safety assessment," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Mar. 04, 2024, doi: 10.1109/TNNLS.2024.3369315.
[58] J. Liu, G. Xu, D. Xiao, L.-Z. Gu, and X.-X. Niu, "A semi-supervised ensemble approach for mining data streams," *J. Comput.*, vol. 8, pp. 2873–2880, 2013.
[59] M. M. Masud, J. Gao, L. Khan, J. Han, and B. Thuraisingham, "A practical approach to classify evolving data streams: Training with limited amount of labeled data," in *Proc. 8th IEEE Int. Conf. Data Mining*, 2008, pp. 929–934.
[60] H. M. Gomes et al., "Adaptive random forests for evolving data stream classification," *Mach. Learn.*, vol. 106, pp. 1469–1495, 2017.
[61] N. C. Oza and S. J. Russell, "Online bagging and boosting," in *Proc. Int. Workshop Artif. Intell. Statist.*, 2001, pp. 229–236.
[62] A. Défossez, L. Bottou, F. Bach, and N. Usunier, "A simple convergence proof of adam and adagrad," 2020, *arXiv:2003.02395*.
[63] T. Pohlert, "The pairwise multiple comparison of mean ranks package (PMCMR)," *R Package*, vol. 9, pp. 2004–2006, 2014.
[64] P. Wang, B. Xue, J. Liang, and M. Zhang, "Feature clustering-assisted feature selection with differential evolution," *Pattern Recognit.*, vol. 140, 2023, Art. no. 109523.

Authorized licensed use limited to: GITAM University. Downloaded on September 10,2025 at 07:15:08 UTC from IEEE Xplore. Restrictions apply.